

CHAPTER - 2

Regular Expression and Finite Automata

2.1 Chomsky Hierarchy of Grammar

- **Language:** A language is a collection of sentences of finite length, all constructed from a finite alphabet of symbols. Thus, the string or sentences is a sequential occurrence of input alphabets or symbols such as 0011.
- **Grammar:** A grammar can be regarded as a device that enumerates the sentences of a language.
- **Formal Grammar:** Formal grammar is a way to describe a formal language i.e. set of finite length strings over a certain finite alphabet. They are named “formal grammar” by the analogy with the concept of grammar of human language.

It is the containment hierarchy of class of formal grammar which was describe by Noam Chomsky in 1956. Here, the automation of the respective grammar is the mathematical model to operate the language of grammar.

<u>Class</u>	<u>Grammars</u>	<u>Languages</u>	<u>Automaton</u>
Type-0	Unrestricted	Recursively enumerable (Turing-recognizable)	Turing machine
Type-1	Context-sensitive	Context-sensitive	Linear-bounded
Type-2	Context-free	Context-free	Pushdown
Type-3	Regular	Regular	Finite

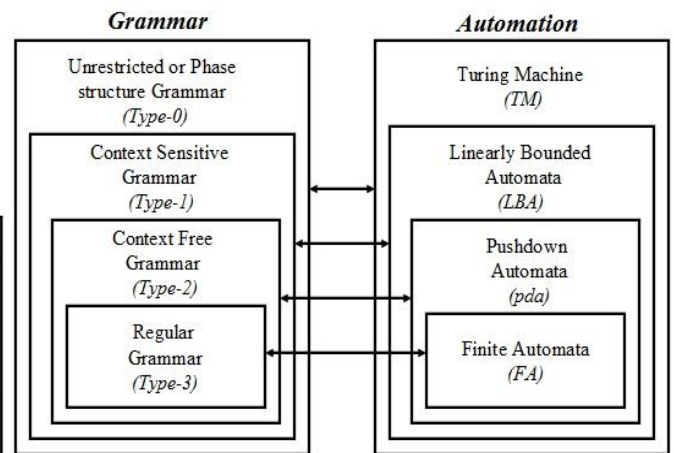


Fig. The Chomsky hierarchy of Grammar

2.2 Regular Expression

The regular expressions are useful for representing certain set of strings in an algebraic fashion. The regular expressions are generated by the **regular grammar** as they are the **language generator**. Actually, the regular expression describes the language accepted by the finite state automata. Any set represented by a regular expression is called a **regular set**. We give a formal definition of regular expression over Σ as follows:

1. Any terminal symbol of the input alphabet such as 0, 1, a, b... $\in$ or $\wedge$ and ϕ are regular expression.
2. The union of two regular expression R1 & R2, written as R1+R2 or R1 $\cup$ R2 is also RE.
3. The concatenation of two regular expression R1 & R2, written as R1.R2 is also RE.
4. The iteration of any regular expression R, written as R* is also regular expression.
5. If R is a regular expression then (R) is also RE. This have different mean for the case of iteration i.e. 1+0* $\neq$ (1+0)*

Identities for Regular Expression

There are some identities for the regular expression that are helpful for simplifying the regular expressions.

- $\Phi + R = R$
- $\Phi R = R \phi = \phi$
- $\wedge R = R \wedge = R$
- $\wedge^* = \wedge$ and $\phi^* = \wedge$
- $R + R = R$

- $R^*R^* = R^*$
- $RR^* = R^*$
- $(R^*)^* = R^*$
- $\wedge + RR^* = R^* = \wedge + R^*R$
- $(PQ)^*P = P(QP)^*$
- $(P + Q)^* = (P^*Q^*)^* = (P^* + Q^*)^*$
- $(P + Q)R = PR + QR$ and $R(P+Q) = RP + RQ$

Equivalence of two regular expressions

Let P and Q be two regular expressions then P & Q are equivalent (we write $P = Q$) iff they represent the same set. Also, P and Q are equivalent iff the corresponding finite automata are equivalent or isomorphic.

Operators of Regular Expression

The regular expression denotes the language. For example, the RE $01^* + 10^*$ denotes the language consisting of all strings that are either a single 0 followed by any number of 1's or a single 1 followed by number of 0's. There are three operations on languages that represent the operators of the regular expression. These operations are:

1. The **union** of two languages L & M denoted by $L+M$, is the set of string that are in either L or M or both. For example, let $L = \{001, 10, 111\}$ and $M = \{\epsilon, 001\}$ then $L + M = \{\epsilon, 10, 001, 111\}$.
2. The **concatenation** of language L & M is the set of string that can be formed by taking any string in L and concatenating it with any string in M and is denoted by $L.M$. For example, let $L = \{001, 10, 111\}$ and $M = \{\epsilon, 001\}$ then $L.M = \{001, 10, 111, 001001, 10001, 111001\}$.
3. The **closure** (or star or Kleene closure) of language L is denoted by L^* and represent the set of those string that can be formed by taking any number of string form L, possibly with repetitions (i.e. the same string may be selected more than once) and concatenating all them. For example, let $L = \{0,1\}$ then L^* is the set of all the string of 0's and 1's with ϵ or $\wedge$, i.e. $L^* = \{\wedge, 0, 1, 01, 10, 11, 000, \dots\}$.

Example

Let $L1 =$ the set of all string of 0's and 1's ending in 00, and then the equivalent regular expression can be described as:

- Any string in $L1$ is obtained by concatenating any string over $\{0, 1\}$ and the string 00. Since, $\{0, 1\}$ is represented by $0 + 1$. Hence $L1$ is represented by $(0 + 1)^*00$.

$L2 =$ the set of all string of 0's and 1's beginning with 0 and ending with 1, and then the equivalent regular expression can be described as:

- Any string in $L2$ is obtained by concatenating 0, any string over $\{0, 1\}$ and 1. Thus $L2$ can be represented by $0(0+1)^*1$.

$L3 = \{\wedge, 11, 1111, 111111 \dots\}$, and then the equivalent regular expression can be described as:

- Any element of $L3$ is either $\wedge$, or a string of even number of 1's so $L3$ can be represented by $(11)^*$.

Notes:

- $\Sigma^* = \Sigma^+ + \epsilon$
- $(0,1)^* = (0+1)^*$ but $(0+1) \neq (0,1)$
- $0+1 =$ Either 0 or 1 but not both. Thus, $0^* + 1^* =$ Either string of zeros or string of one.
- $01 =$ First consume 0 then 1. Thus, $0^*1^* =$ First string of zeros then the string of one.

Decision properties of regular language

1. Is the language describe is empty?
2. Is the particular string 'w' is in the described language 'L'?
3. Is two described language are equivalent or describe the same language?

A. Testing of emptiness of Regular Language

If there is any path from the start state to some accepting state, then the language is non-empty, while if the accepting states are all separated from the start state, then the language is empty. Thus, if we can compute the set or any one of reachable states or accepting state from the start state by reading the any input string then we answer "NO" (i.e. the language is non-empty) and otherwise we answer "YES" (i.e. the language is empty).

B. Testing membership in a Regular Language

The next important question is that if 'w' is a given string represented explicitly and 'L' be a regular language represented by an automata or regular expression then "Is 'w' is in 'L' or not?" For this, we have to construct a DFA and then simulate the DFA processing the string of input symbol 'w', beginning in the start state. If the DFA end in the accepting state, the answer is "YES", otherwise the answer is "NO".

C. Testing of equivalence of two Regular Languages

Another important question is "whether the different description of two languages indicate the same things or not?" Since, the different description of a language is given by a tool called regular expression. To check the equivalence of regular languages, we construct the DFA of different description. If we get the same DFA for different description, the answer is "YES", otherwise the answer is "NO".

2.3 Automata Theory

- Automata theory: the study of abstract computing devices, or "machines"
- Before computers (1930), **Alan Turing** studied an abstract machine (*Turing machine*) that had all the capabilities of today's computers (concerning what they could compute). His goal was to describe precisely the boundary between what a computing machines could do and what it could not do.
- Simpler kinds of machines (*finite automata*) were studied by a number of researchers and useful for a variety of purposes. Theoretical developments bear directly on what computer scientists do today
 - Finite automata, formal grammars: design/ construction of software
 - Turing machines: help us understand what we can expect from a software
 - Theory of intractable problems: are we likely to be able to write a program to solve a given problem? Or we should try an approximation, a heuristic...

Why Study Automata Theory?

Finite automata are a useful model for many important kinds of software and hardware:

1. Software for designing and checking the behavior of digital circuits
2. The lexical analyzer of a typical compiler, that is, the compiler component that breaks the input text into logical units
3. Software for scanning large bodies of text, such as collections of Web pages, to find occurrences of words, phrases or other patterns
4. Software for verifying systems of all types that have a finite number of distinct states, such as communications protocols of protocols for secure exchange information

Finite state Machine (FSM)

A finite state machine or simply a state machine is a particular conceptualization of a sequential circuit composed of inputs & outputs, a finite number of states, transition between those states and actions. Thus, a computer can be viewed as huge FSMs. In other word, a finite state machine defined as a system of automation where energy, material and information are transformed and use for performing some functions without direct participation of man. Examples are: automatic machines, and automatic photo printing machine. In computer science the term 'automation' means 'discrete automation' and is defined in more abstract way as shown in figure.

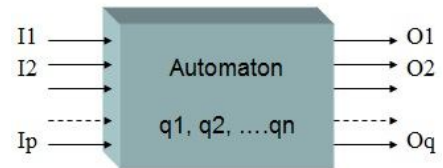
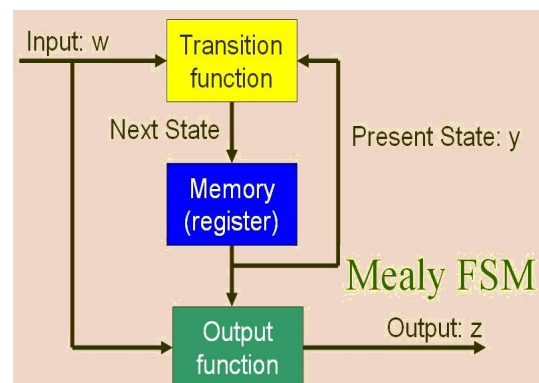
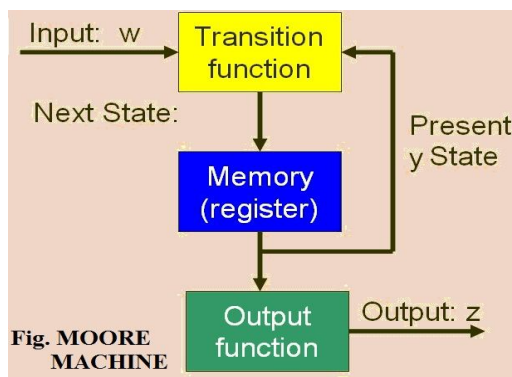


Fig. Model of Discrete Automation

Formally, the finite state machine $M = (Q, \Sigma, O, \delta, q_0, g)$ is a 6-tuple that consists of :

- Q = Finite non-empty set of states.
 - Σ = Finite non-empty set of input alphabets.
 - O = Finite non-empty set of output alphabets.
 - δ = State relation or transition function.
 - q_0 = Start state, $q_0 \in Q$.
 - g = Output function for all pair of states.
- * An automation in which the output depends only on the input is called *automation without a memory*.
- * An automation, in which the output depends on input and the state as well, is called *automation with finite memory*.
- * An automation in which the output depends only on state of the machine is called *Moore Machine*.
- * An automation in which the output depends on the state as well as on the input at any instant of time is called a *Mealy Machine*.



Finite State Automata (FSA)

Finite state automata (singular Automation) are a special kind of finite state machine with no outputs but have a set of final states. They recognize string that take the starting state to final state but no storage mechanism so sometime called the automation without memory. The finite state automata can be used as **language recognizers**. This application plays a fundamental role in the design and *construction of compilers for programming languages*.

The sets of strings recognized by finite automata are called regular expression. The sets built up from the null set, the empty string, and singleton (a set by single digit) strings by concatenations, unions and Kleene closures, in arbitrary order is called regular sets.

Analytically, a finite automaton $M = (Q, \Sigma, \delta, q_0, F)$ is a 5-tuple which consists of

Q = Finite non-empty set of states

Σ = Finite non-empty set of input alphabet

δ = Transition function that assigns a next state and inputs (since, $Q \times \Sigma \rightarrow \delta$)

q_0 = Initial state or start state ($q_0 \in Q$)

F = Subset of Q (i.e. $F \subseteq Q$) consisting of final state (or accepting states). It is assumed that there may be more than one final state.

The various components of the FSA or FA are

- **Input tape:** The input tape is divided into squares, each square containing a single symbol from the input alphabet Σ . The absence of the end mark on the input string indicates the infinite length of string.
- **Reading Head:** The reading head examines only one square at a time and can move one square left or right (but generally from left to right).
- **Finite Control:** Here, all the states of the system on reading the input string remain on this set so it is called the finite control. It controls the activities on the system as follows:
 - Motion of the reading head along the tape to the next square.
 - The next state of the finite automata after the processing of $\delta(q, a)$, where q = state of machine, a = input symbol & δ = transition function.

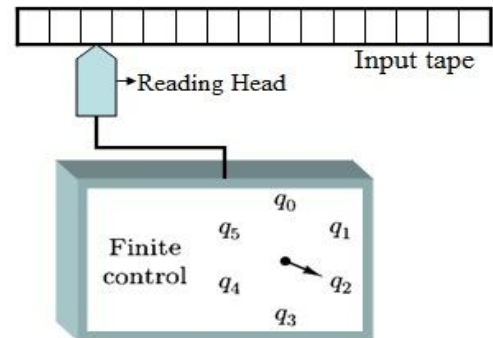


Fig. B.D. of a Finite automata

→ **State:** A uniquely identifiable set of values measured at various points in a digital system.

→ **Next State**

- The state to which the state machine makes the next transition, determined by the inputs present when the device is clocked.
- At the termination of the input string the state of the FA must reach to any one final state among the various possible final states.

Notation for FA

There are two preferred notations for describing the automata, which are:

A. State Diagram or Transition Diagram or graph

A transition diagram or transition system is a finite directed weighted graph in which each vertex (or node) represents a state and the directed edge indicates the transition of the state and the edges are labelled with input. The *initial state* is represented by a circle or bubble with an arrow pointed towards it, the *final state* by two concentric circles and other states by just a circle.

The state diagram accepts the string w in Σ^* if there exists a path which originates from some initial state, goes along the arrows and terminates at some final state with path value w .

Example:

Let us have the transition function as: $(q_0, 0) \rightarrow q_0$, $(q_0, 1) \rightarrow q_1$, $(q_1, 0) \rightarrow q_0$, $(q_1, 1) \rightarrow q_1$, with the initial state q_0 and the final state q_1 , then the corresponding state diagram is given by:

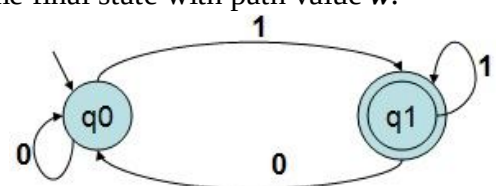


Fig. Transition Diagram

B. State Table or Transition Table

A transition table is a conventional tabular representation of a function like δ that take *two arguments and returns a value*. The row of the table corresponds to the status and the column corresponds to the inputs. The entry for the row corresponding to state 'q' and the column correspond to input 'a' is the state $\delta(q, a)$.

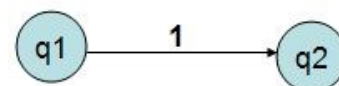
Q/ Σ	0	1
$\rightarrow q_0$	q_0	q_0
q_1	q_0	q_0

Fig. Transition Table

Types of Finite Automata

A. Deterministic Finite Automata (DFA)

DFA is a FA that is in a single state after reading any sequence of inputs. The term 'deterministic' refer to the fact that on each input, there is only one state to which the automata can transition from its current states.



A deterministic finite automaton $M = (Q, \Sigma, \delta, q_0, F)$ is a 5-tuple which consists of
 Q = Finite non-empty set of states

Σ = Finite non-empty set of input alphabet

δ = Transition function (usually called the *direct transition function*) that assign a next states and inputs (since, $Q \times \Sigma \rightarrow \delta$). In the transition diagram representation of DFA, δ is represented by arcs between states and the label on the arc. If 'q' is a state, and 'a' is an input symbol then $\delta(q, a)$ is represent that state 'p' such that there is an arc labelled 'a' from 'q' to 'p'.

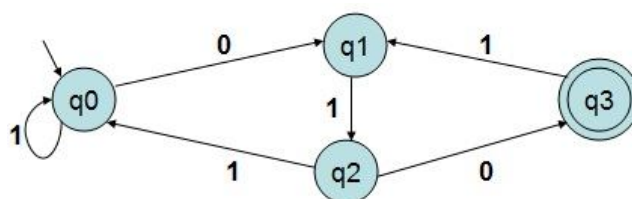


Fig. Transition Diagram for DFA

q_0 = Initial states or start states ($q_0 \in Q$)

F = Subset of Q (i.e. $F \subseteq Q$) consisting of final state (or accepting states). It is assumed that there may more than one final state.

Language of DFA

The language of DFA, $A = (Q, \Sigma, \delta, q_0, F)$ is defined by $L(A) = \{ W : \delta(q_0, W) \text{ is in } F \}$ i.e. the language of 'A' is the set of string 'W' that takes the start state q_0 to one of the accepting state. If language 'L' is $L(A)$ for some DFA 'A', then we say 'L' is a regular language.

Acceptability of a string by a DFA

A string 'x' is accepted by a DFA, $A = (Q, \Sigma, \delta, q_0, F)$ if $\delta(q_0, x) = q$ for some $q \in F$.

Q/ Σ	0	1
$\rightarrow q_0$	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

Fig. Transition Table

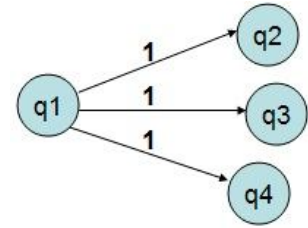
Hence

$$q_0 \xrightarrow{1} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_0$$

$$\begin{aligned} \delta_{q_0, 110101} &= \delta_{q_1, 10101} \\ &= \delta_{q_0, 0101} \\ &= \delta_{q_2, 101} \\ &= \delta_{q_3, 01} \\ &= \delta_{q_1, 1} \\ &= \delta_{q_0, \wedge} \\ &= q_0 \end{aligned}$$

B. Non-deterministic Finite Automata (NFA)

Non-deterministic means a choice of moves for an automata i.e. rather than prescribing a unique move in each situation of input, we allow a set of possible moves. As shown in figure the state from q1 can change to the possible states q2, q3 & q4 on reading the same input 1. As there are multiple state for the same input so the system sometime may not reach to the final state, hence termed as the non-deterministic automata.



A non-deterministic finite automaton $M = (Q, \Sigma, \delta, q_0, F)$ is a 5-tuple which consists of
 Q = Finite non-empty set of states

Σ = Finite non-empty set of input alphabet

q_0 = Initial states or start states ($q_0 \in Q$)

F = Subset of Q (i.e. $F \subseteq Q$) consisting of final state (or accepting states). It is assumed that there may more than one final state.

δ = Transition function that assign a next states and inputs (since, $Q \times \Sigma \rightarrow \delta$).

Notice that the only difference between NFA & DFA is that the δ returns a set of value in case of NFA and a single value in case of DFA.

Conversion of NFA to DFA

- For every NFA, there exists a DFA which simulate the behavior of NFA. Alternatively, if L is the accepted by NFA, then there exists a DFA which also accept L .
- Every state of a DFA will be represented by some subset of set of states of NFA and therefore the transition of NFA to DFA is normally called a **subset construction**. If there are 'n' states of NFA then the possible subsets are 2^n , in which some states are used for construction of the equivalent DFA. Thus, for two states q_0 & q_1 for NFA then its possible subsets are $2^2 = 4$ [i.e. $\{\emptyset\}, \{q_0\}, \{q_1\}, \{q_0q_1\}$]
- For converting the NFA to DFA,
 - Seek all the transition from start state for every symbol in set of inputs ' Σ '. If you get the different state beside the start state then terms them as a new state.
 - For all the new state, check all the transition for every symbol in Σ . Repeat step-2 till we are getting a new state.
 - All these state which consists at least one accepting or final state of given NFA will be considered as the final state for the equivalent DFA.

EXERCISE – I

NFA with ϵ Transition

If a FA is modified to permit transition without input symbol, along with zero, one or more transition on input symbols, then we get a NFA with ϵ - transitions because the transition mode without symbol are called ϵ -transitions. The idea of the ϵ -NFA come from the concept of closure or iteration.

Formally we represent an ϵ -NFA by $A = (Q, \Sigma, \delta, q_0, F)$, where all components have their same interpretation as for NFA except δ is now a function that takes as arguments: -

- All the states lie in Q after transition.
- A member of $\Sigma \cup \{\epsilon\}$ i.e. either an input symbol or the symbol for empty string ϵ and it cannot be the member of the alphabet Σ only.

The figure shows the NFA with ϵ -transition because it is possible to make the transition from state q_0 to q_1 and q_1 to q_3 without consuming any of the input symbols. This state diagram accept the string 010 as it can be written as $0\epsilon 1\epsilon 0$.

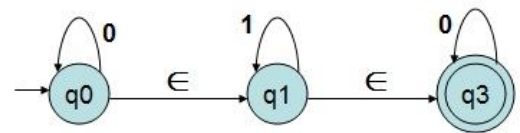
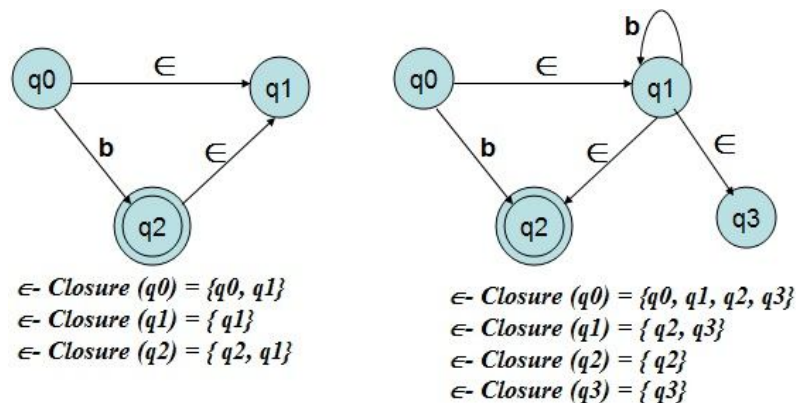


Fig. ϵ -NFA

ϵ -Closure

A string 'w' in Σ will be accepted by ϵ -NFA if there exist at least one path corresponding 'w', which start in an initial state and ends in one of the final states. Since this path may be formed by ϵ -transitions as well as non- ϵ -transitions. We may require defining a function ϵ -Closure (q), where q is a state of the automata.

The function ϵ -Closure (q) is defined as "the set of all those states of the automata (i.e. ϵ -NFA) which can be reached from q on a path labeled by ϵ i.e. without consuming any input symbols. Note that if there is no ϵ for any state to transit then the ϵ -Closure (q) function for that state will be the same state.



Conversion of ϵ -NFA to DFA

This conversion process follows two steps:

- Find the ϵ -Closure (q) function of each states
- Identify the transition function of each state obtained from ϵ -Closure (q) function.

EXERCISE – II

Conversion of Regular Expression (RE) into equivalent Deterministic Finite Automata (DFA)

Theorem: Every regular expression 'R' can be recognized by a transition system. In other word, for every string 'w' in the set of regular expression 'R', there exists a path from the initial state to a final state with path value 'w'.

Since, any string generated by regular expression is accepted by finite state automata. Thus, on reading the string from start to finish, it is accepted by the FSA in regular order of path.

It should be noted that in the case of iteration, we can add new state by performing the null or $\wedge$ move and

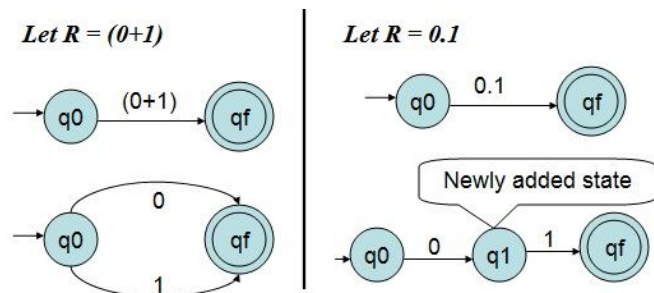


Fig. Conversion of RE in to equivalent FA

generally we merge the $\wedge$ - move in final step of conversation process. For the RE to FA conversation, the order of precedence is *, (, +, & . and the associatively moves from left to right.

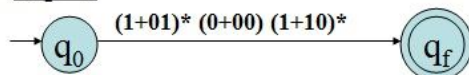
Note: The representation of a given regular expression may have multiple **isomorphic graphs** in conversion of equivalent deterministic finite automata.

Example

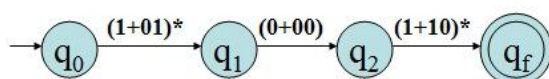
The given regular expression (R) = $(1+01)^* (0+00) (1+10)^*$

Now, the equivalent Finite Automata (FA) can be constructed as:

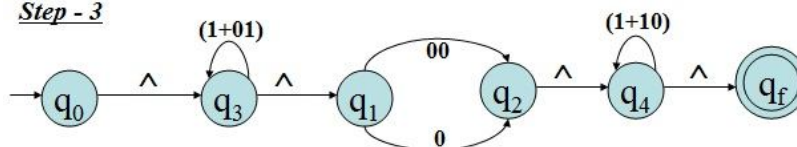
Step - 1



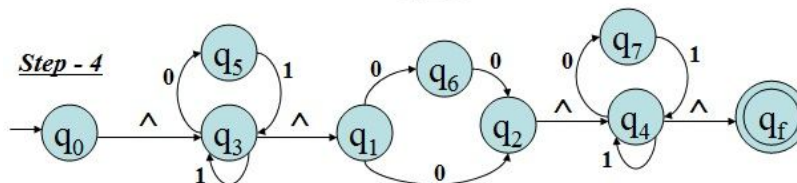
Step - 2



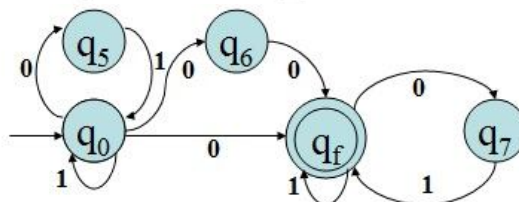
Step - 3



Step - 4



Step - 5



Q/Σ	0	1
→ q ₀	q ₅ , q ₆ , q _f	q ₀
q ₅	φ	q ₀
q ₆	q _f	φ
q ₇	φ	q _f
q _f	q ₇	q _f

Fig. Transition Table

Since, the resulting Finite Automata is NFA, which can be converted to the DFA as follow. Let δ' be the new transition function for the equivalent DFA.

Thus,

$$\delta' (q_0, 0) = \delta (q_0, 0) = \{q_5, q_6, q_f\} \text{ (New state)} \rightarrow q_r \text{ (Say)}$$

$$\delta' (q_0, 1) = \delta (q_0, 1) = \{q_0\}$$

$$\delta' [(q_5, q_6, q_f), 0] = \delta (q_5, 0) \cup \delta (q_6, 0) \cup \delta (q_f, 0) = \{\phi\} \cup \{q_f\} \cup \{q_7\} = \{q_7, q_f\} \text{ (New state)} \rightarrow q_s \text{ (Say)}$$

$$\delta' [(q_5, q_6, q_f), 1] = \delta (q_5, 1) \cup \delta (q_6, 1) \cup \delta (q_f, 1) = \{q_0\} \cup \{\phi\} \cup \{q_f\} = \{q_0, q_f\} \text{ (New state)} \rightarrow q_t \text{ (Say)}$$

$$\delta' [(q_7, q_f), 0] = \delta (q_7, 0) \cup \delta (q_f, 0) = \{\phi\} \cup \{q_7\} = \{q_7\} \text{ (New state)}$$

$$\delta' [(q_7, q_f), 1] = \delta (q_7, 1) \cup \delta (q_f, 1) = \{q_f\} \cup \{q_f\} = \{q_f\} \text{ (New state)}$$

$\delta' [(q_0, q_f), 0] = \delta (q_0, 0) \cup \delta (q_f, 0) = \{q_5, q_6, q_f\} \cup \{q_7\} = \{q_5, q_6, q_7, q_f\}$ (New state) $\rightarrow q_u$ (Say)

$\delta' [(q_0, q_f), 1] = \delta (q_0, 1) \cup \delta (q_f, 1) = \{q_0\} \cup \{q_f\} = \{q_0, q_f\}$

$\delta' (q_7, 0) = \delta (q_7, 0) = \{\phi\}$

$\delta' (q_7, 1) = \delta (q_7, 1) = \{q_f\}$

$\delta' (q_f, 0) = \delta (q_f, 0) = \{q_7\}$

$\delta' (q_f, 1) = \delta (q_f, 1) = \{q_f\}$

$\delta' [(q_5, q_6, q_7, q_f), 0] = \delta (q_5, 0) \cup \delta (q_6, 0) \cup \delta (q_7, 0) \cup \delta (q_f, 0) = \{\phi\} \cup \{q_f\} \cup \{\phi\} \cup \{q_7\} = \{q_7, q_f\}$

$\delta' [(q_5, q_6, q_7, q_f), 1] = \delta (q_5, 1) \cup \delta (q_6, 1) \cup \delta (q_7, 1) \cup \delta (q_f, 1) = \{q_0\} \cup \{\phi\} \cup \{q_f\} \cup \{q_f\} = \{q_0, q_f\}$

Thus, the new transition δ' of the DFA can be represented by the transition table as:

Now,

i. *Acceptance of string 0011100*

$\delta (q_0, 0011100) = \delta (q_r, 011100)$
 $= \delta (q_s, 11100)$
 $= \delta (q_t, 1100)$
 $= \delta (q_f, 100)$
 $= \delta (q_f, 00)$
 $= \delta (q_7, 0)$
 $= \delta (\phi, \wedge)$

Q/ Σ	0	1
$\rightarrow q_0$	q_r	q_0
q_r	q_s	q_t
q_s	q_7	q_f
q_t	q_u	q_t
q_7	ϕ	q_f
q_f	q_7	q_f
q_u	q_s	q_t

This implies that the string is not acceptable by the Finite Automata.

ii. *Acceptance of string 10111001101*

$\delta (q_0, 10111001101) = \delta (q_0, 0111001101)$
 $= \delta (q_r, 111001101)$
 $= \delta (q_t, 11001101)$
 $= \delta (q_t, 1001101)$
 $= \delta (q_t, 001101)$
 $= \delta (q_u, 01101)$
 $= \delta (q_s, 1101)$
 $= \delta (q_f, 101)$
 $= \delta (q_f, 01)$
 $= \delta (q_7, 1)$
 $= \delta (q_f, \wedge)$
 $= \{q_f\}$

This implies that the string is acceptable by the Finite Automata.

EXERCISE – III

Conversion of Deterministic Finite Automata (DFA) into equivalent Regular Expression (RE)

Arden's Theorem: Let P & Q be two regular expressions over Σ . If P does not contain $\wedge$, then the following equation in R , namely $R = Q + RP$ has a unique solution (i.e. one and only one solution) given by $R = QP^*$.

Steps for DFA to RE conversion

1. Define the equation for each state. The equation of start state must contain the $\wedge$ also.
2. Attempt to find the expression contain only the alphabet (as RE contain the alphabets only) for the final state. If there is more than one final state then identify the expression of alphabets only for all final state and finally merge them into one expression.

Example

For the given Transition table, the Finite Automata $M = [\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, \{q_3\}]$ where δ can be defined as:

Now, FA can be constructed as:

Now, the equivalent Regular Expression (R.E.) of the Finite Automata (F.A.) can be constructed as:

Q/Σ	0	1
→q0	q1	q0
q1	q2	q0
q2	q3	q0
q3	q3	q3

Fig. Transition Table

Step1: Defining the equation for each state with reference to the input symbol (Only incoming arrows)

Here,

$$q_0 = q_0.1 + q_1.1 + q_2.1 + \wedge \quad \text{---- I}$$

$$q_1 = q_0.0 \quad \text{---- II}$$

$$q_2 = q_1.0 \quad \text{---- III}$$

$$q_3 = q_2.0 + q_3.0 + q_3.1 \quad \text{---- IV}$$

Step2: Finding the expression of symbol only for Final state

Here, From equation IV, we have

$$q_3 = q_2.0 + q_3.0 + q_3.1$$

$$q_3 = q_2.0 + q_3(0 + 1)$$

Using the Arden's theorem, we have

$$q_3 = q_2.0(0 + 1)^* \quad \text{---- V}$$

Now, using equation II & III on the equation V, we have

$$q_3 = q_0.000(0 + 1)^* \quad \text{---- VI}$$

Now, from equation I, II, & III, we have

$$q_0 = q_0.1 + q_0.01 + q_0.001 + \wedge$$

$$q_0 = q_0(1 + 01 + 001) + \wedge$$

Using the Arden's theorem, we have

$$q_0 = \wedge(1 + 01 + 001)^* = (1 + 01 + 001)^* \quad \text{---- VII}$$

Now, from equation VI, & VII, we have

$$q_3 = (1 + 01 + 001)^* 000(0 + 1)^*$$

This is the R.E. which is accepted by FA.

EXERCISE – IV

Closure Property of Regular set

The closure property of the regular set is defined with reference to the Finite Automata (FA) i.e. the language acceptor. Class of language accepted by finite automata is closed under: Union, concatenation, Kleene closure or Star, complementation and Intersection.

A. Union

Let $M_1 = (Q_1, \Sigma_1, \delta_1, q_1, F_1)$ be NFA that accept a regular language $L = L(M_1)$ and $M_2 = (Q_2, \Sigma_2, \delta_2, q_2, F_2)$ be another NFA that accept a regular language $L = L(M_2)$. Here, we assume that Q_1 and Q_2 are two disjoint sets.

Now, we construct NFA $M = (Q, \Sigma, \delta, q, F)$, such that it can accepts $L = L(M_1) \cup L(M_2)$, where q is the new state not in Q_1 or Q_2 . Where,

- $Q = Q_1 \cup Q_2 \cup \{q\}$
- $\Sigma =$ Set of input states $= \Sigma_1 \cup \Sigma_2 \cup \{\wedge\}$
- $\delta = \delta_1 \cup \delta_2 \cup [(q, \wedge) \rightarrow q_1, (q, \wedge) \rightarrow q_2]$
- $q =$ Start state.
- $F = F_1 \cup F_2$.

M begins any computation by non-deterministically choosing to either the initial state of M_1 or the initial state of M_2 and interpreter, M initiates either M_1 or M_2 . Formally, if $w \in \Sigma^*$,

then $(q, w) \xrightarrow{*}_M (P, \epsilon)$, for some $P \in F$ if and only if $(q, w) \xrightarrow{*}_{M_1} (P, \epsilon)$, for some $P \in F_1$
, or $(q, w) \xrightarrow{*}_{M_2} (P, \epsilon)$, for some $P \in F_2$

Hence, $L(M) = L(M_1) \cup L(M_2)$.

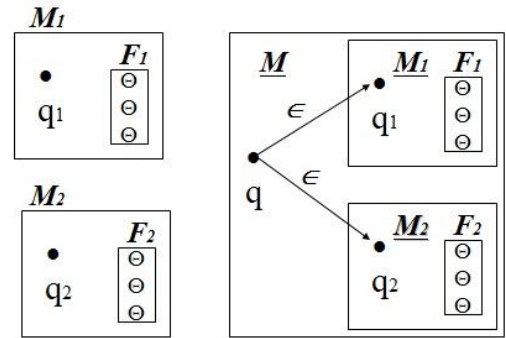


Fig. Regular Language is closed under union

B. Concatenation

Let $M_1 = (Q_1, \Sigma_1, \delta_1, q_1, F_1)$ be NFA that accept a regular language $L = L(M_1)$ and $M_2 = (Q_2, \Sigma_2, \delta_2, q_2, F_2)$ be another NFA that accept a regular language $L = L(M_2)$. Now, we construct NFA $M = (Q, \Sigma, \delta, q, F)$, such that it can accepts $L = L(M) = L(M_1) \bullet L(M_2)$.

Where,

- $Q = Q_1 \cup Q_2$
- $\Sigma =$ Set of input states $= \Sigma_1 \cup \Sigma_2 \cup \{\wedge\}$
- $\delta = \delta_1 \cup \delta_2 \cup [(F_1, \wedge) \rightarrow q_2]$
- $q =$ Start state $= q_1$
- $F = F_2$.

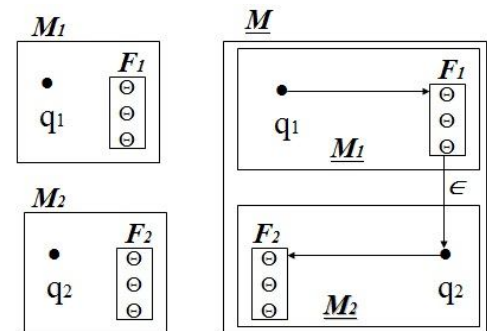


Fig. RL is closed under Concatenation

Formally, $(q, w) \xrightarrow{*}_M (P, \epsilon)$, for some $P \in F$ if and only if there are $w_1, w_2 \in \Sigma^*$ and there is a $P_1 \in F_1$, such that $w = w_1 w_2$ and $(q_2, w_2) \xrightarrow{*}_{M_2} (P_2, \epsilon)$ & $(q_1, w_1) \xrightarrow{*}_{M_1} (P_1, \epsilon)$

Hence, $L(M) = L(M_1) \bullet L(M_2)$.

C. Kleene closure

Let $M_1 = (Q_1, \Sigma_1, \delta_1, q_1, F_1)$ be NFA that accept a regular language $L = L(M_1)$. Now, we construct NFA $M = (Q, \Sigma, \delta, q, F)$, such that it can accepts $L = L(M) = L(M_1)^*$.

Where,

- $Q = Q_1 \cup \{q\}$
- $\Sigma =$ Set of input states $= \Sigma_1 \cup \{\wedge\}$
- $\delta = \delta_1 \cup [(q, \wedge) \rightarrow q_1, (q_1, \wedge) \rightarrow F_1, (F_1, \wedge) \rightarrow q]$
- $q =$ Start state
- $F = F_1 \cup \{q\}$.

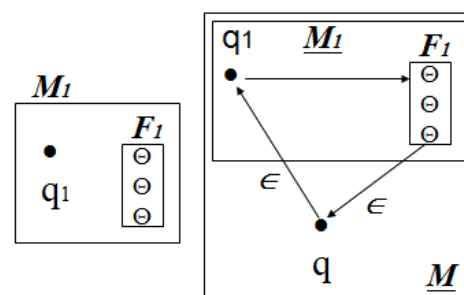


Fig. RL is closed under Kleene Closure

Here, M consists of the states of M_1 and all the transition of M_1 and also, any final state of M_1 is the final state of M . In addition, M has a new initial state 'q'. The new initial state is also final so that ϵ is accepted. From it, the state q_1 can be reached on the input ϵ , so that the operations of M_1 can be initiated after M has been started in state 'q'. Finally, the transition on input ϵ are added from each final state of M_1 back to q_1 , once a string in $L(M_1)$ has been read, computation can resume from the initial state of M_1 .

It follows the inspection of M that if $w \in L(M)$ then either $w = \wedge$ or $w = w_1 w_2 \dots w_k$, for $k \geq 1$

i. for $i = 1, 2 \dots k$, there is $F_i \in F$ such that $(q_1, w) \xrightarrow{*}_{M_1} (F_1, \epsilon)$

Hence, $w \in L(M_1)^*$ so $L(M) = L(M_1)^*$.

Pumping Lemma for Regular Set or Language

- The pumping lemma for regular languages describes an essential property of all regular languages. Informally, it says that all sufficiently long words in a regular language may be pumped - that is, have a middle section of the word repeated an arbitrary number of times - to produce a new word which also lies within the same language.
- The pumping lemma gives a method of pumping (generating) many input string from a given string. In general, it gives a necessary condition for an input string to belong to a regular set.
- We have established that the class of language known as regular language for at least four different descriptions. They are the language accepted by DFA, NFA, ϵ -NFA, and regular expression. Not every language is regular. The pumping lemma is a powerful technique for showing certain language is not-regular.
- Let $\Sigma = \{0, 1\}^* = \{0, 1, 00, 11, 01, 001, \dots\}$ be the set of string generated from the given input symbol. These set of the string may fall in different class of language as shown in figure. Thus, the pumping lemma helps us to identify whether the set of string or language fall in the class of regular language or not. We can use proof by contradiction together with the Pumping Lemma to demonstrate a language is not regular.
- Pumping lemma is the relation between the set of string (n) and the length of the string (m or $|w|$).

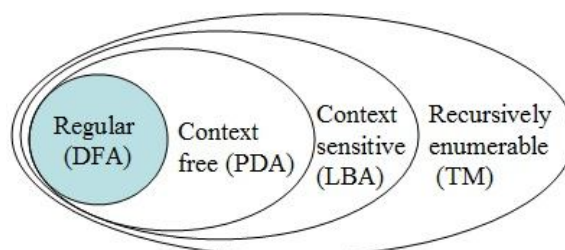


Fig. The Chomsky hierarchy of Grammar

- Neither the original nor the general version of the pumping lemma gives a sufficient condition for a language to be regular. That is, the lemma holds for some non-regular languages. If the lemma does not hold for a language L , then L is not regular. If the lemma does hold for a language L , L might be regular.

Statement

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a finite automaton with n states that accepts a regular language L . Let w be any string such that $w \in L$ and $|w| \geq n$ then there exists x, y, z such that

1. $w = xyz$,
2. $y \neq \epsilon$ (Given assumption)
3. $|xy| \leq n$ ($|xy| = n$ when $z = \epsilon$) and
4. $xy^iz \in L$, for all $i \geq 0$.

Here,

- n = Number of states.
- w = Any string $w \in L$
- $|w| = m$ = Length of string in w (if $w = abcd$, $|w| = 4$.)
- y^i = Read i at infinite times.

Proof

Let $w = a_1, a_2, a_3 \dots a_m$, (m is length of string)

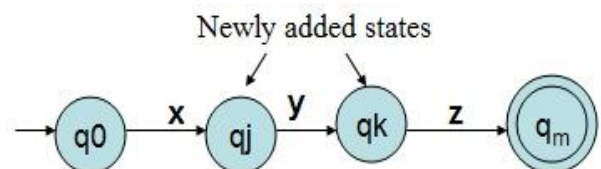
$\delta(q_0, a_1, a_2, a_3 \dots a_i) = q_i$, ($i = 1, 2, 3 \dots n$) = Sequence of states in the path with path value 'w'.

$Q' = \{q_0, q_1, q_2 \dots q_n\}$

Now, the input string 'w' can be decomposed into three substrings as:

- $x = a_1, a_2, a_3 \dots a_j$
- $y = a_{j+1}, \dots a_k$
- $z = a_{k+1}, \dots a_m$

Since, we know that for any regular expression, there exists a path from initial state to final state with path value the given regular expression. Thus, for 'w' the finite automata can be constructed as shown in figure.

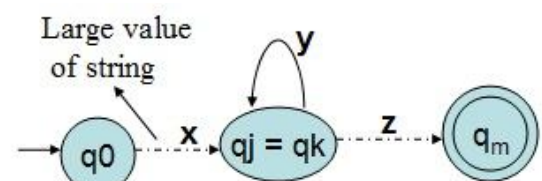


Here, String size $|w| = 3$ but the states = 4

Here, on reading the string xyz (i.e. $|w| = m = 3$), a new states are added on the existing states (i.e. states (n) = 4). But by the definition of pumping lemma, we have, $|w| \geq n$. Thus, by using *Pigeon-Hole* principle, there must coincide at least two states in Q , as there are only n distinct states defined but on applying the input string the states becomes $n+1$. Thus, among various pair of repeated states, we take first as q_j and q_k (i.e. $q_j = q_k$) for merging & hence the path with value w in the transition diagram of M is shown in figure below.

$xy^iz \in L$, for all $i \geq 0$.

- When $i = 0$, $w = xy$ (String is recognized by finite automata)
- When $i = 1$, $w = xyz$ (String is recognized by finite automata)
- When $i = 2$, $w = xy^2z$ (String is recognized by finite automata) and so on. Hence we can conclude that this condition is valid for all regular string.



Here, String size $|w| = 3$ but the states = 3

The figure shows the string accepted by the finite automata M. Here, M starts for the initial state q_0 and on applying the string x , it reaches the state $q_j = q_k$. Again, on applying y , it comes back to the same state $q_j = q_k$ until application of y^i , $i \geq 0$ and finally it reaches to q_m on applying z . Thus, the string of 'y' can pump in several numbers of times to generate large string. Hence, $xy^iz \in L$.

For $|xy| \leq n$

The verification of the case is obvious by the help of the given FA defined for the regular string 'w'. Since, $w = xyz = a_1, a_2 \dots a_j, a_{j+1} \dots a_k, a_{k+1} \dots a_m$, thus we can write $1 \leq j < k \leq m$ and this implies that $|xy| \leq n$.

Hence, we can say that the pumping lemma is proved.